



Semester Project Report

Sehat Saathi

A Hospital Management System for GIKI

Submitted By:

Hassan Khalid	2023242
Jagtar Singh	2023266
Muhammad Ibrahim	2023446
Mehdi Raza	2023466

Submitted To:

Sir Said Nabi

Faculty of Computer Science

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Contents

1 Project Title: Sehat Saathi - Hospital Management System	2
1.1 Team Members	2
2 Project Overview	2
3 Technology Stack	2
4 Entity-Relationship Diagram (ERD)	3
5 Database Schema	4
6 Functionalities Implemented	6
6.1 Doctor Module	6
6.2 Patient Module	6
6.3 Administrator Module	7
6.4 Receptionist Module	7
7 GitHub & Jira Details	8
8 Challenges Faced	9
8.1 Technical Challenges	9
8.2 Project Management Challenges	9
8.3 Domain-Specific Challenges	10
9 Future Enhancements	10
9.1 Advanced Features	10
9.2 Technical Improvements	10
9.3 Integration Possibilities	11
10 Conclusion	11

1 Project Title: Sehat Saathi - Hospital Management System

1.1 Team Members

Name	Roll Number	Contribution
Muhammad Ibrahim	2023446	Backend APIs, DB design
Jagtar Singh	2023266	Frontend, Documentation
Mehdi Raza	2023466	Jira Management, Backend
Hassan Khalid	2023242	Frontend, Documentation

2 Project Overview

Sehat Saathi serves as an integrated platform connecting doctors, patients, administrators, and receptionists, facilitating seamless information flow and process automation within the hospital environment. The system aims to:

- Streamline hospital operations through digital transformation
- Enhance patient-doctor interactions
- Improve record-keeping and data accessibility
- Automate routine administrative tasks
- Provide real-time access to medical information

3 Technology Stack

The project utilizes the following technologies:

- **Frontend:** React.js with Material-UI components for responsive design
- **Backend:** Django REST Framework for robust API development
- **Database:** PostgreSQL for reliable data storage and retrieval
- **Project Management:** Jira for agile development tracking
- **Code Hosting:** GitHub for version control and collaboration
- **Authentication:** The project uses session-based authentication rather than JWT tokens.
- **AI Chatbot:** Gemini 1.5

4 Entity-Relationship Diagram (ERD)

The database design consists of the following entities:

1. hms_app_department
2. hms_app_doctor
3. hms_app_patient
4. hms_app_receptionist
5. hms_app_appointment
6. hms_app_labtest
7. hms_app_medicalrecord
8. hms_app_bill

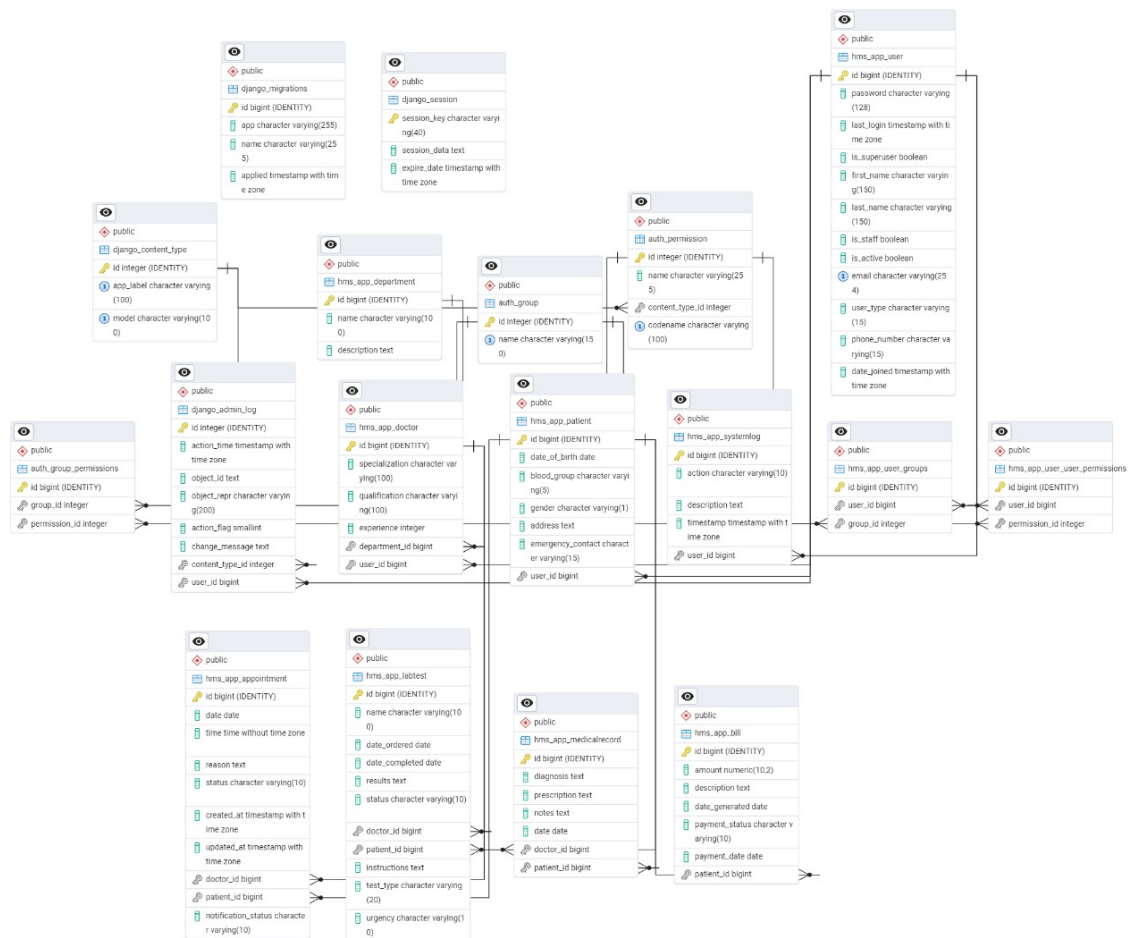


Figure 1: Entity Relationship Diagram

5 Database Schema

SQL Schema Definition

```
-- Table: hms_app_department
CREATE TABLE hms_app_department (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT
);

-- Table: hms_app_doctor
CREATE TABLE hms_app_doctor (
    id SERIAL PRIMARY KEY,
    user_id INTEGER UNIQUE NOT NULL, -- Foreign key to auth_user
    specialization VARCHAR(100) NOT NULL,
    qualification VARCHAR(200) NOT NULL,
    experience INTEGER NOT NULL,
    department_id INTEGER NOT NULL, -- Foreign key to hms_app_department
    FOREIGN KEY (user_id) REFERENCES auth_user(id) DEFERRABLE INITIALLY DEFERRED,
    FOREIGN KEY (department_id) REFERENCES hms_app_department(id) DEFERRABLE INITIALLY DEFERRED
);

-- Table: hms_app_patient
CREATE TABLE hms_app_patient (
    id SERIAL PRIMARY KEY,
    user_id INTEGER UNIQUE NOT NULL, -- Foreign key to auth_user
    date_of_birth DATE NOT NULL,
    blood_group VARCHAR(5) NOT NULL,
    gender VARCHAR(1) NOT NULL,
    address TEXT NOT NULL,
    emergency_contact VARCHAR(20) NOT NULL,
    FOREIGN KEY (user_id) REFERENCES auth_user(id) DEFERRABLE INITIALLY DEFERRED
);

-- Table: hms_app_appointment
CREATE TABLE hms_app_appointment (
    id SERIAL PRIMARY KEY,
    date DATE NOT NULL,
    time TIME WITHOUT TIME ZONE NOT NULL,
    reason TEXT,
    status VARCHAR(20) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL,
    doctor_id INTEGER NOT NULL, -- Foreign key to hms_app_doctor
    patient_id INTEGER NOT NULL, -- Foreign key to hms_app_patient

```

```
notification_status VARCHAR(10),
instructions TEXT,
test_type VARCHAR(20),
urgency VARCHAR(10),
FOREIGN KEY (doctor_id) REFERENCES hms_app_doctor(id) DEFERRABLE INITIALLY DEFERR
FOREIGN KEY (patient_id) REFERENCES hms_app_patient(id) DEFERRABLE INITIALLY DEFE
);

-- Table: hms_app_labtest
CREATE TABLE hms_app_labtest (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  date_ordered DATE,
  date_completed DATE,
  results TEXT,
  status VARCHAR(20),
  patient_id INTEGER NOT NULL, -- Foreign key to hms_app_patient
  doctor_id INTEGER NOT NULL,
  FOREIGN KEY (patient_id) REFERENCES hms_app_patient(id) DEFERRABLE INITIALLY DEFE
  FOREIGN KEY (doctor_id) REFERENCES hms_app_doctor(id) DEFERRABLE INITIALLY DEFERR
);

-- Table: hms_app_medicalrecord
CREATE TABLE hms_app_medicalrecord (
  id SERIAL PRIMARY KEY,
  diagnosis TEXT,
  prescription TEXT,
  notes TEXT,
  date DATE NOT NULL,
  doctor_id INTEGER NOT NULL, -- Foreign key to hms_app_doctor
  patient_id INTEGER NOT NULL, -- Foreign key to hms_app_patient
  FOREIGN KEY (doctor_id) REFERENCES hms_app_doctor(id) DEFERRABLE INITIALLY DEFERR
  FOREIGN KEY (patient_id) REFERENCES hms_app_patient(id) DEFERRABLE INITIALLY DEFE
);

-- Table: hms_app_bill
CREATE TABLE hms_app_bill (
  id SERIAL PRIMARY KEY,
  amount NUMERIC(10,2) NOT NULL,
  description TEXT,
  status VARCHAR(20) NOT NULL,
  date_generated DATE NOT NULL,
  payment_status VARCHAR(10),
  payment_date DATE,
  patient_id INTEGER NOT NULL, -- Foreign key to hms_app_patient
  FOREIGN KEY (patient_id) REFERENCES hms_app_patient(id) DEFERRABLE INITIALLY DEFE
);
```

6 Functionalities Implemented

6.1 Doctor Module

- **Profile Access (via ID):** Manage personal profile, contact information, specialization, and department
- **Patient Medical Record Access:** Access complete patient medical records with search and filter capabilities
- **Lab Test Ordering:** Order lab tests directly through the system with automated notifications
- **Appointment Management:** View and manage daily/weekly appointment schedules

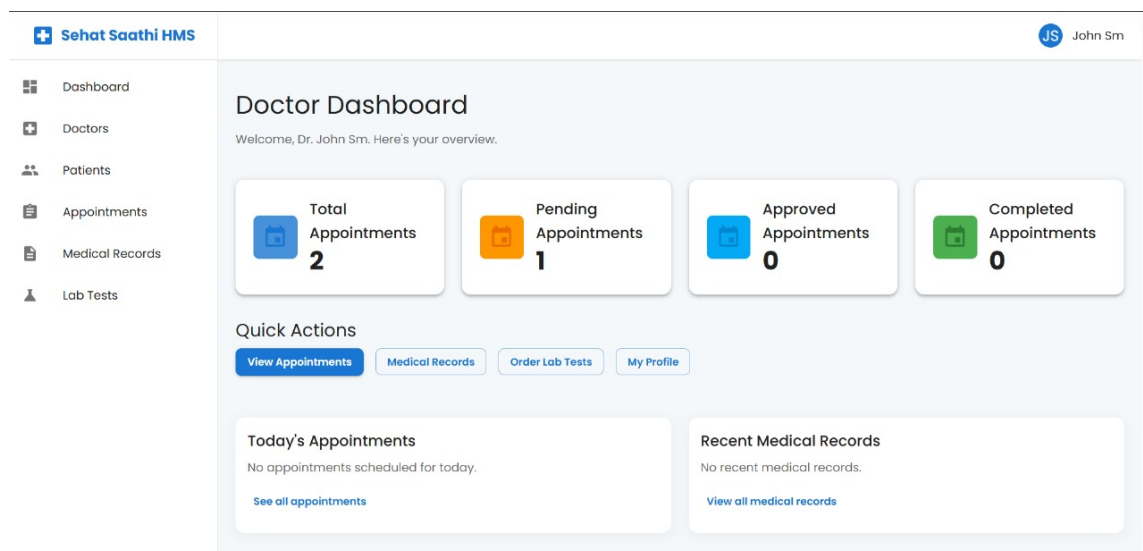


Figure 2: Doctor Dashboard

6.2 Patient Module

- **Appointment Booking:** Book appointments with preferred doctors based on availability
- **Available Doctor View:** View list of doctors and their specializations with ratings and reviews
- **Appointment Reminders:** Receive automated reminders about upcoming appointments via email/SMS
- **Medical Record Access:** Access personal medical records and download reports in PDF format

- **Financial History View:** View financial history, including bills and payments with filtering options

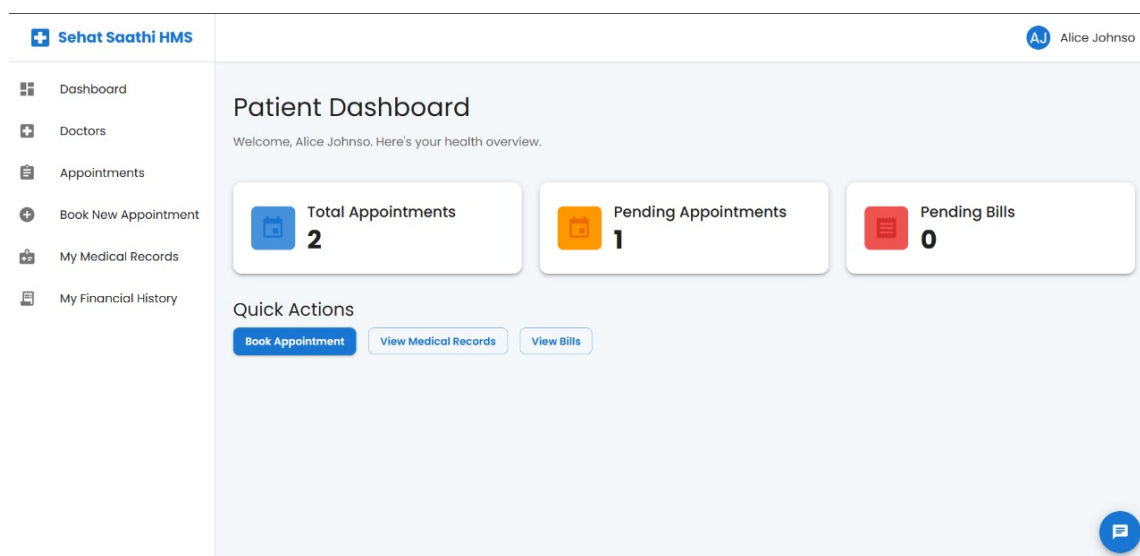


Figure 3: Patient Dashboard

6.3 Administrator Module

- **User Management:** Perform CRUD operations on user accounts with role assignment
- **Role-Based Permission Management:** Assign specific permissions to different roles (doctor, patient, receptionist)
- **System Logs:** View system logs to track activities and errors with timestamp and user information
- **Statistics and Reporting:** Generate reports and view system statistics with data visualization

6.4 Receptionist Module

- **Patient Registration:** Register new patients into the system with all necessary details
- **Appointment Management:** Create, read, update, and delete appointments with conflict detection
- **Bill and Invoice Generation:** Generate bills and invoices for patients with multiple payment options

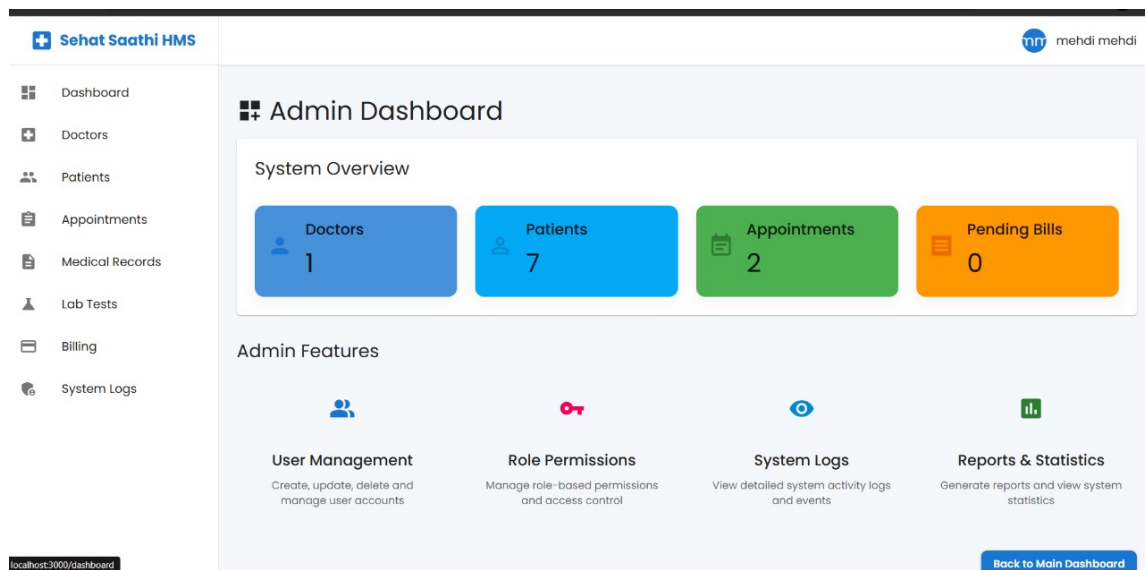


Figure 4: Administrator Dashboard

- **Available Doctor View:** View list of available doctors for scheduling with real-time updates
- **Patient Profile Updates:** Update patient profiles with new information and medical history

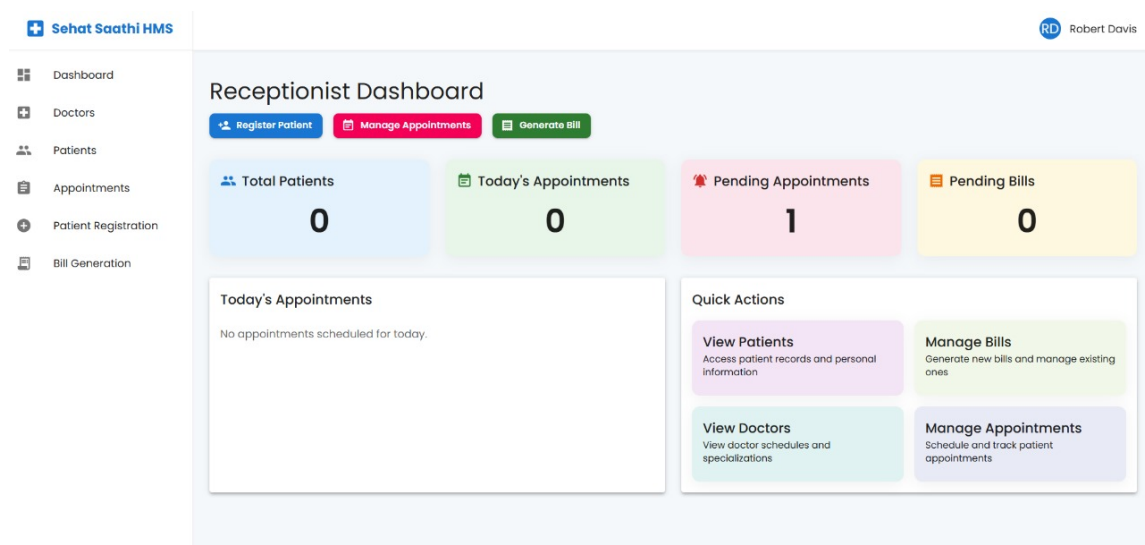


Figure 5: Receptionist Dashboard

7 GitHub & Jira Details

Our project is accessible through the following links:

- **Main Repository:** <https://github.com/jagtar5/SehatSathi>

- **Sub Repository 1:** <https://github.com/MuhammadMehdiRaza/Hospital-Management-System>
- **Sub Repository 2:** <https://github.com/mIBRAHIM707/Sehat-Saathi>
- **Jira Project Board:** <https://giki-team-disj6rql.atlassian.net/jira/software/projects/TN/boards/67>

The GitHub repository contains all source code, documentation, and setup instructions for the Sehat Saathi HMS. The Jira board tracks our development progress, sprint planning, and issue management following an Agile methodology.

8 Challenges Faced

During the development of Sehat Saathi HMS, our team encountered several significant challenges:

8.1 Technical Challenges

- **Database Design Complexity:** Creating an efficient schema that accommodated the complex relationships between patients, doctors, appointments, and medical records required multiple iterations to optimize for performance and data integrity.
- **Authentication & Authorization:** Implementing a secure role-based access control system that properly restricted sensitive patient data while allowing appropriate access for medical staff was particularly challenging.
- **Real-time Appointment Scheduling:** Developing a conflict-free appointment scheduling system that could handle concurrent requests and provide immediate feedback required sophisticated database transaction handling.

8.2 Project Management Challenges

- **Scope Management:** The initial project scope was ambitious, necessitating careful prioritization of features based on core functionality requirements.
- **Integration Issues:** Coordinating the integration between frontend React components and backend Django APIs required establishing clear interface definitions and extensive testing.
- **Learning Curve:** Team members had varying levels of experience with the technology stack, requiring additional time for knowledge sharing and skill development.

8.3 Domain-Specific Challenges

- **Medical Data Privacy:** Ensuring compliance with data privacy principles while maintaining system functionality and performance.
- **Complex Workflows:** Accommodating the various hospital workflows and exception cases that emerged during requirement gathering sessions with stakeholders.

9 Future Enhancements

Based on our implementation experience and stakeholder feedback, we have identified several promising areas for future enhancement:

9.1 Advanced Features

- **Telemedicine Integration:** Implement video consultation capabilities to facilitate remote patient-doctor interactions, especially beneficial for follow-up appointments and minor consultations.
- **Pharmacy Management Module:** Extend the system to include a pharmacy management component for medication tracking, prescription fulfillment, and inventory management.
- **Business Intelligence Dashboard:** Develop advanced analytics and reporting tools to provide hospital management with actionable insights on operational efficiency, resource utilization, and patient outcomes.
- **Mobile Application:** Create dedicated mobile applications for patients and doctors to improve accessibility and engagement with the system.

9.2 Technical Improvements

- **Machine Learning Integration:** Implement ML algorithms for predictive analytics, such as patient admission forecasting and resource optimization.
- **Electronic Health Record (EHR) Standards Compliance:** Enhance the medical records module to comply with international EHR standards for better interoperability with other healthcare systems.
- **Containerization & Microservices:** Refactor the architecture to utilize containerization technologies like Docker and adopt a microservices approach for improved scalability and maintenance.
- **Advanced Security Features:** Implement additional security measures such as two-factor authentication, enhanced audit logging, and automated security scanning.

9.3 Integration Possibilities

- **Insurance Provider Integration:** Develop APIs for direct communication with insurance providers for claim processing and verification.
- **Laboratory Information System (LIS) Integration:** Connect with laboratory systems for seamless lab test ordering and result reporting.
- **External Healthcare Provider APIs:** Facilitate information exchange with other healthcare providers for comprehensive patient care.

10 Conclusion

The development of Sehat Saathi Hospital Management System represents a significant step toward modernizing and improving healthcare delivery at GIKI. Through our implementation, we have addressed critical operational challenges by providing a comprehensive digital solution that connects all stakeholders in the healthcare ecosystem.

Our system successfully delivers on its primary objectives of streamlining hospital operations, improving patient-doctor interactions, enhancing record-keeping, automating administrative tasks, and providing real-time access to medical information. The modular design allows for continued expansion and adaptation to evolving healthcare needs.

The project has demonstrated several key achievements:

- Creation of a robust, secure platform for managing sensitive medical information
- Implementation of intuitive interfaces tailored to the specific needs of doctors, patients, administrators, and receptionists
- Development of a flexible architecture that can accommodate future integrations and enhancements
- Application of database design principles to efficiently model complex healthcare relationships and workflows

Through this project, our team has gained valuable experience in full-stack development, database design, user experience considerations, and project management in the context of healthcare information systems. We have also developed a deeper appreciation for the intricate challenges involved in digitizing healthcare processes while maintaining security, privacy, and usability.

As we conclude this phase of development, we look forward to the continued evolution of Sehat Saathi HMS based on user feedback and emerging healthcare technology trends. We believe this system has established a strong foundation that will serve GIKI's healthcare needs effectively while providing opportunities for continuous improvement and innovation.